Part 1

i. Design file (.v) for the Ripple Carry Adder

```
module RCA_4bits(
    input clk,
    input enable,
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [4:0] Q, // outputs for reg
    output enable_led
);

    // temp variables
    wire c1, c2, c3;
    wire [4:0] sum;

    // 4 full adders
    full_adder FA0 (.A(A[0]), .B(B[0]), .Cin(Cin), .S(sum[0]), .Cout(c1));
    full_adder FA1 (.A(A[1]), .B(B[1]), .Cin(c1),   .S(sum[1]), .Cout(c2));
    full_adder FA2 (.A(A[2]), .B(B[2]), .Cin(c2),   .S(sum[2]), .Cout(c3));
    full_adder FA3 (.A(A[3]), .B(B[3]), .Cin(c3),   .S(sum[3]), .Cout(sum[4]));

    // for the reg logic
    register_logic REG (
        .clk(clk),
        .enable(enable),
        .Data(sum),
        .Q(Q)
    );

    // turn on led if button pressed
    assign enable_led = enable;

endmodule

module full_adder(
    input A, B, Cin,
    output S, Cout
);

    assign S = A ^ B ^ Cin;
    assign Cout = (A & B) | (A & Cin) | (B & Cin);

endmodule
```

```
module register_logic(
    input clk,
    input enable,
    input [4:0] Data,
    output reg [4:0] Q
);

    always @(posedge clk) begin
        if (enable) begin
            Q <= Data; //the nonblocking statement
        end
    end

endmodule
```

ii. Test-bench file (.v)

```
module tb_RCA_4bits;

    reg clk;
    reg enable;
    reg [3:0] A;
    reg [3:0] B;
    reg Cin;

    wire [4:0] Q;
    wire enable_led;

    RCA_4bits dut (.clk(clk), .enable(enable), .A(A), .B(B), .Cin(Cin),
        .Q(Q), .enable_led(enable_led));

    //make clk
    always #5 clk = ~clk;

    initial begin
        clk = 0;
        enable = 0;
        A = 0;
        B = 0;
        Cin = 0;

        // reset time/init time
        #20;

        //want enable always on
```

```
        enable = 1;


        A = 4'b0001; B = 4'b0101; Cin = 1'b0;
        #20;

        A = 4'b0111; B = 4'b0111; Cin = 1'b0;
        #20;

        A = 4'b1000; B = 4'b0111; Cin = 1'b1;
        #20;

        A = 4'b1100; B = 4'b0100; Cin = 1'b0;
        #20;

        A = 4'b1000; B = 4'b1000; Cin = 1'b1;
        #20;

        A = 4'b1001; B = 4'b1010; Cin = 1'b1;
        #20;

        A = 4'b1111; B = 4'b1111; Cin = 1'b0;
        #20;

        $finish;
    end
endmodule
```

iii. Complete Table 1 from the simulation

A[3:0]	B[3:0]	Cin	Sum[3:0]	Cout
0001	0101	0	0110	0
0111	0111	0	1110	0
1000	0111	1	0000	1
1100	0100	0	0000	1
1000	1000	1	0001	1
1001	1010	1	0100	1
1111	1111	0	1110	1

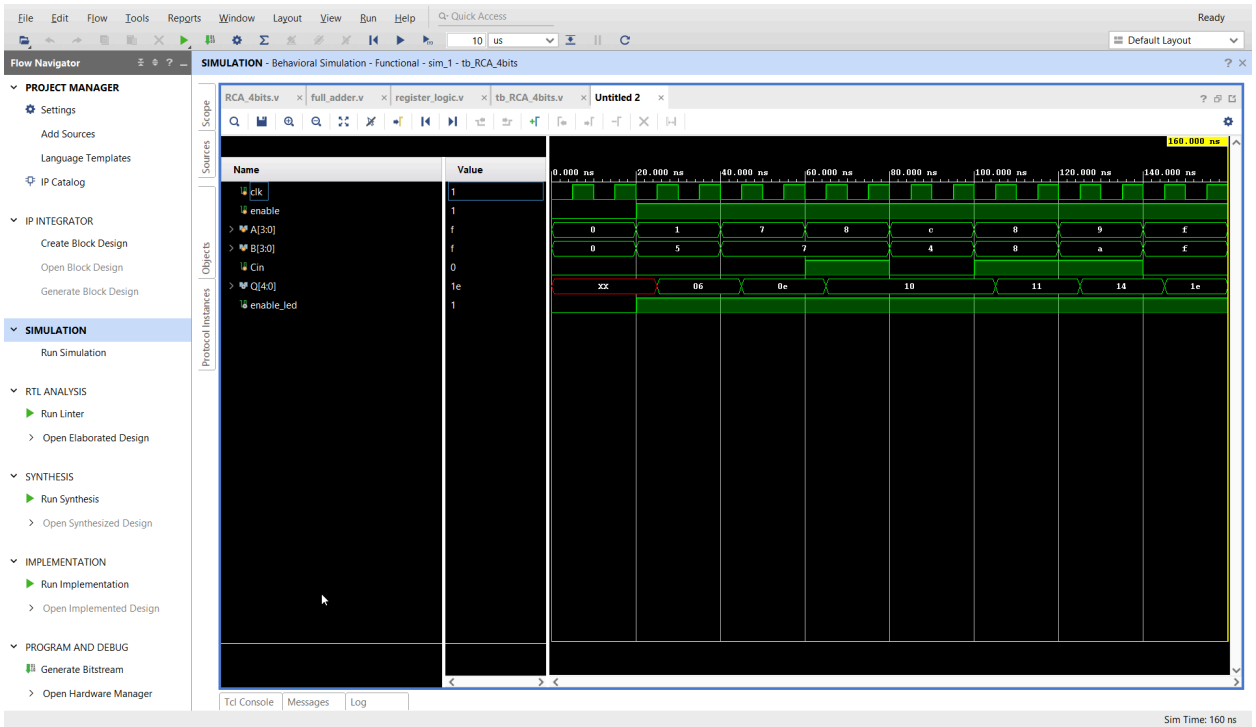
iv. Constraints File (Just the uncommented portion)

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
```

```
	set_property IOSTANDARD LVCMOS33 [get_ports clk]
	create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]

## Switches
## A[3:0] mapped to SW[3:0]
set_property PACKAGE_PIN V17 [get_ports {A[0]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
set_property PACKAGE_PIN V16 [get_ports {A[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
set_property PACKAGE_PIN W16 [get_ports {A[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]
set_property PACKAGE_PIN W17 [get_ports {A[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[3]}]

## B[3:0] mapped to SW[7:4]
set_property PACKAGE_PIN W15 [get_ports {B[0]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
set_property PACKAGE_PIN V15 [get_ports {B[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
set_property PACKAGE_PIN W14 [get_ports {B[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
set_property PACKAGE_PIN W13 [get_ports {B[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]

## Cin mapped to SW[8]
set_property PACKAGE_PIN V2 [get_ports Cin]
	set_property IOSTANDARD LVCMOS33 [get_ports Cin]


## LEDs
## Q[4:0] mapped to LD[4:0]
set_property PACKAGE_PIN U16 [get_ports {Q[0]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[0]}]
set_property PACKAGE_PIN E19 [get_ports {Q[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[1]}]
set_property PACKAGE_PIN U19 [get_ports {Q[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[2]}]
set_property PACKAGE_PIN V19 [get_ports {Q[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[3]}]
set_property PACKAGE_PIN W18 [get_ports {Q[4]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[4]}]

## enable_led mapped to LED L1 (LD15 on the board)
```

```
set_property PACKAGE_PIN L1 [get_ports enable_led]
        set_property IOSTANDARD LVCMOS33 [get_ports enable_led]


## Buttons
## enable mapped to button U18 (Center Button)
set_property PACKAGE_PIN U18 [get_ports enable]

        set_property IOSTANDARD LVCMOS33 [get_ports enable]
```

v. Simulation waveform for the above test-cases



Part 2

vi. All the equations for Ci's and Si's

```
C[0] = Cin;
C[1] = G[0] | (P[0] & C[0]);
C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & C[0]);
C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & C[0]);
C[4] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) | (P[3] & P[2] & P[1] & G[0]) | (P[3] & P[2]
& P[1] & P[0] & C[0]);

Sum[0] = P[0] ^ C[0];
Sum[1] = P[1] ^ C[1];
Sum[2] = P[2] ^ C[2];
Sum[3] = P[3] ^ C[3];
```

vii. Design files (.v) for the Carry Lookahead Adder and Register Logic

```
module CLA_4bits(
    input clk,
    input enable,
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [4:0] Q, //load registers, should contain the 4 num bits
    output enable_led
);

    wire [3:0] G, P, S; //Generate, Propagate, Sum
    wire [4:0] C; //Carry
    wire [3:0] Sum;
    wire Cout;
    assign C[0] = Cin;

    assign P = A ^ B; // propagate
    assign G = A & B; // generate

    //carry bits
    assign C[1] = G[0] | (P[0] & C[0]);
    assign C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & C[0]);
    assign C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & C[0]);
    assign C[4] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) | (P[3] & P[2] & P[1] & G[0]) |
(P[3] & P[2] & P[1] & P[0] & C[0]);

    //sum is propagate xor with carry
    assign Sum = P ^ C[3:0];
    assign Cout = C[4];

    //reg logic
    register_logic reg_inst (
        .clk(clk),
        .enable(enable),
        .Data({Cout, Sum}), // combine cout and sum
        .Q(Q)
    );

    //turn led on when switch for enable is pressed
    assign enable_led = enable;

endmodule
```

```
module register_logic(
    input clk,
    input enable,
    input [4:0] Data,
    output reg [4:0] Q
);

    always @(posedge clk) begin
        if (enable) begin
            Q <= Data; //the nonblocking statement
        end
    end

endmodule
```

viii. Test-bench file (.v)

```
module tb_CLA_4bits;

    // vars
    reg clk;
    reg enable;
    reg [3:0] A;
    reg [3:0] B;
    reg Cin;

    wire [4:0] Q;
    wire enable_led;

    // dut
    CLA_4bits dut (.clk(clk), .enable(enable), .A(A), .B(B), .Cin(Cin),
        .Q(Q), .enable_led(enable_led));

    always #5 clk = ~clk;

    initial begin
        // initial vals
        clk = 0;
        enable = 0;
        A = 0;
        B = 0;
        Cin = 0;

        #20; //reset/init delay
```

```
        enable = 1; // we want to always update

        A = 4'b0000; B = 4'b0101; Cin = 1'b0; #20;

        A = 4'b0101; B = 4'b0111; Cin = 1'b0; #20;

        A = 4'b1000; B = 4'b0111; Cin = 1'b1; #20;

        A = 4'b1001; B = 4'b0100; Cin = 1'b0; #20;

        A = 4'b1000; B = 4'b1000; Cin = 1'b1; #20;

        A = 4'b1101; B = 4'b1010; Cin = 1'b1; #20;

        A = 4'b1110; B = 4'b1111; Cin = 1'b0; #20;

        $finish;
    end
endmodule
```

ix. Complete Table 2 from the simulation

A[3:0]	B[3:0]	Cin	Sum[3:0]	Cout
0000	0101	0	0101	0
0101	0111	0	1100	0
1000	0111	1	0000	1
1001	0100	0	1101	0
1000	1000	1	0001	1
1101	1010	1	1000	1
1110	1111	0	1101	1

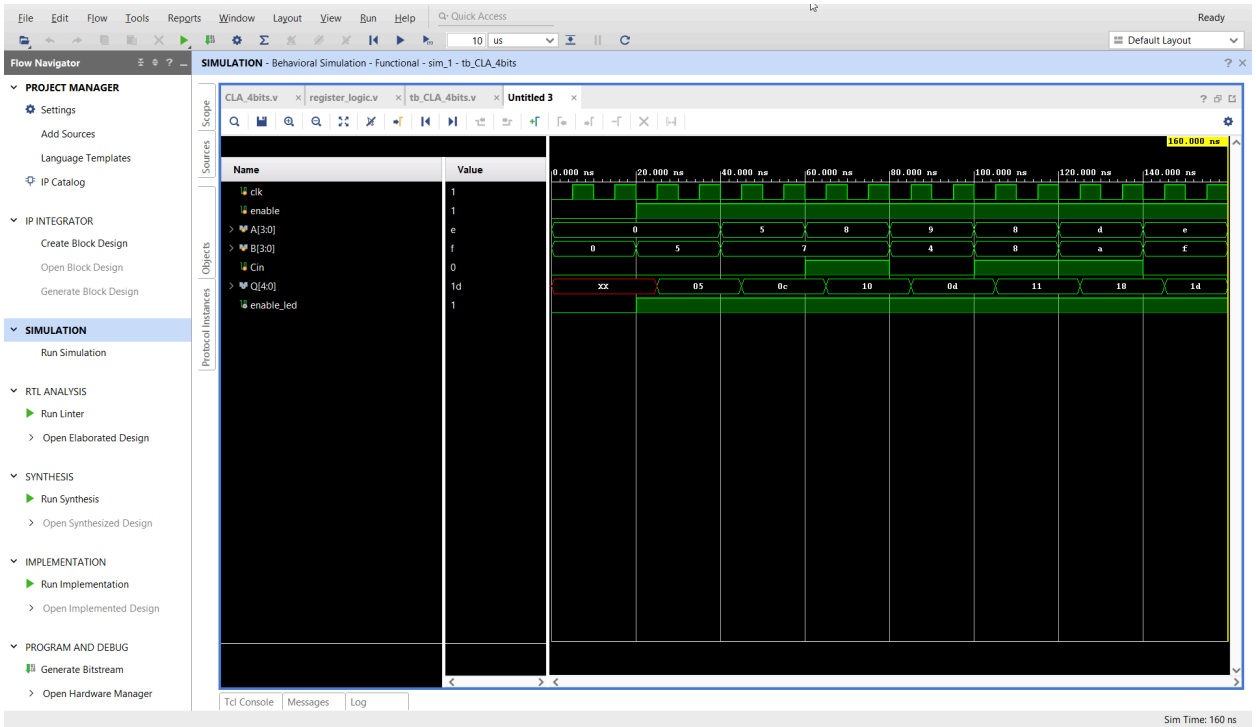
x. Constraints File (Just the uncommented portion)

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]

        set_property IOSTANDARD LVCMOS33 [get_ports clk]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
clk]

## Switches
## A[3:0] mapped to SW[3:0]
set_property PACKAGE_PIN V17 [get_ports {A[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
```

```
set_property PACKAGE_PIN V16 [get_ports {A[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
set_property PACKAGE_PIN W16 [get_ports {A[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]
set_property PACKAGE_PIN W17 [get_ports {A[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {A[3]}]

## B[3:0] mapped to SW[7:4]
set_property PACKAGE_PIN W15 [get_ports {B[0]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
set_property PACKAGE_PIN V15 [get_ports {B[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
set_property PACKAGE_PIN W14 [get_ports {B[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
set_property PACKAGE_PIN W13 [get_ports {B[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]

## Cin mapped to SW[8]
set_property PACKAGE_PIN V2 [get_ports Cin]
	set_property IOSTANDARD LVCMOS33 [get_ports Cin]


## LEDs
## Q[4:0] mapped to LD[4:0]
set_property PACKAGE_PIN U16 [get_ports {Q[0]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[0]}]
set_property PACKAGE_PIN E19 [get_ports {Q[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[1]}]
set_property PACKAGE_PIN U19 [get_ports {Q[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[2]}]
set_property PACKAGE_PIN V19 [get_ports {Q[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[3]}]
set_property PACKAGE_PIN W18 [get_ports {Q[4]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {Q[4]}]

## enable_led mapped to LED L1 (LD15 on the board)
set_property PACKAGE_PIN L1 [get_ports enable_led]
	set_property IOSTANDARD LVCMOS33 [get_ports enable_led]


## Buttons
## enable mapped to button U18 (Center Button)
set_property PACKAGE_PIN U18 [get_ports enable]
```

set_property IOSTANDARD LVCMOS33 [get_ports enable]

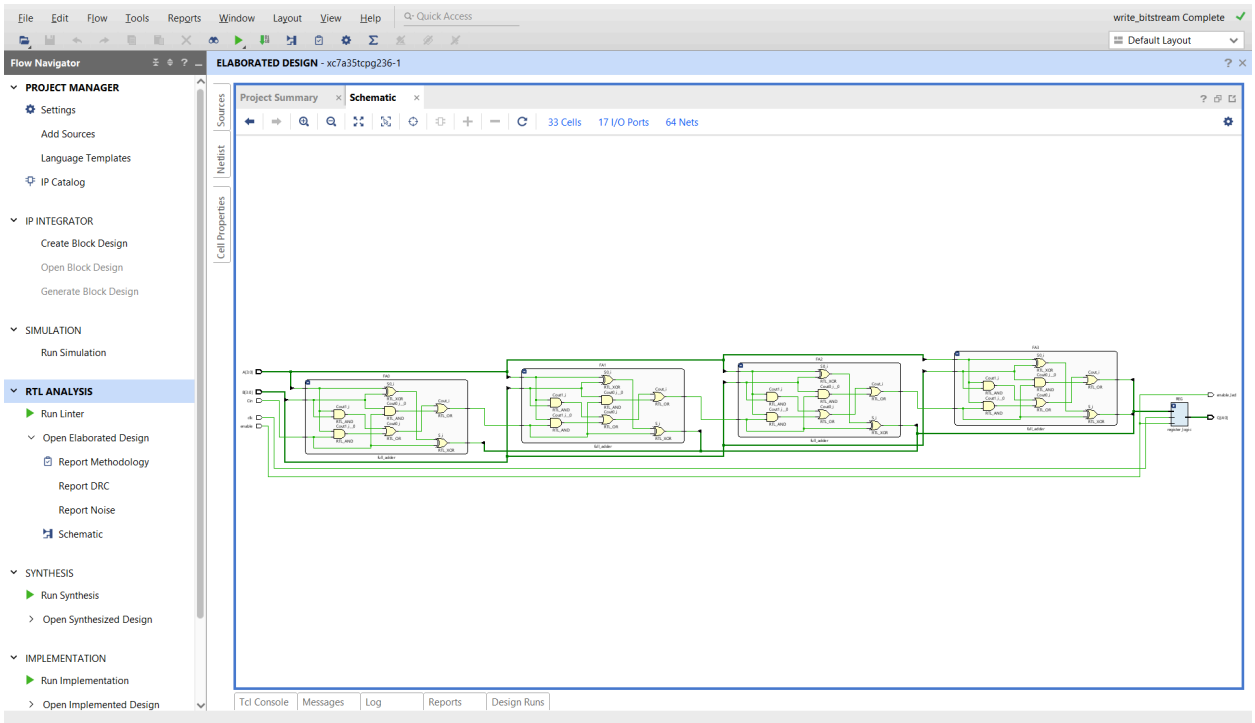
xi. Simulation waveform for the above test-cases



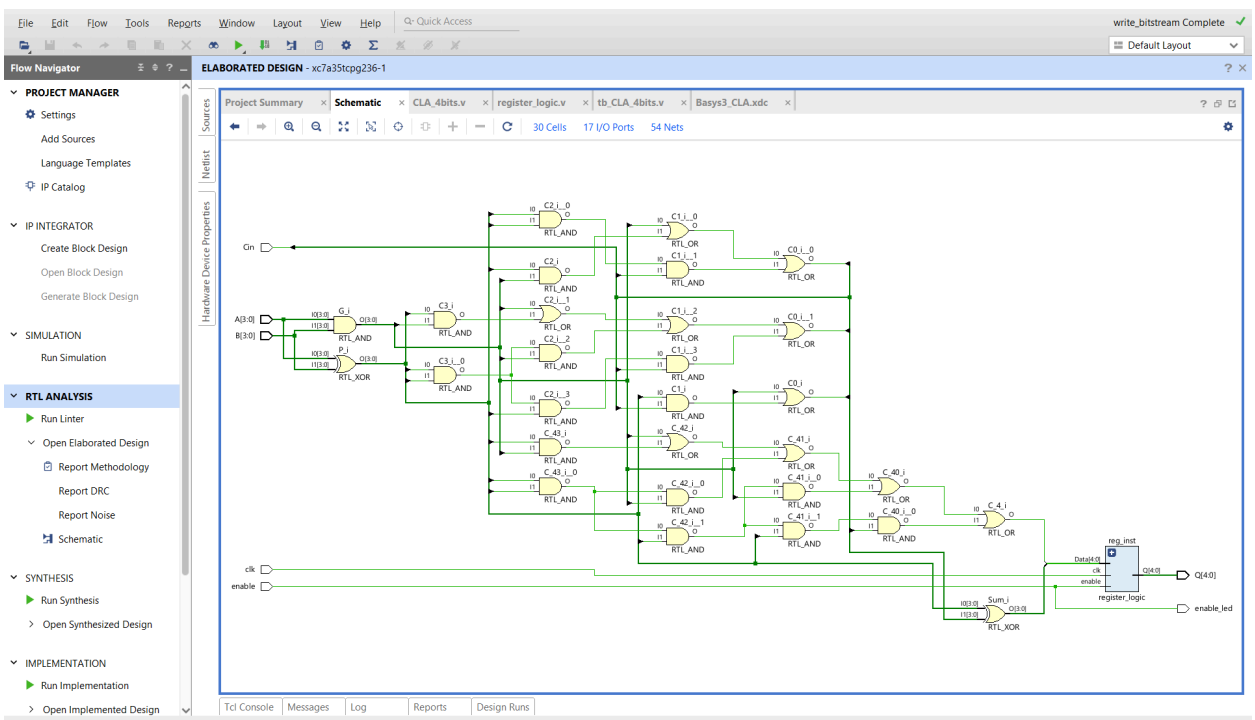
Part 3

xii. Screenshots of the gate-level schematics for both the adder techniques

RCA:



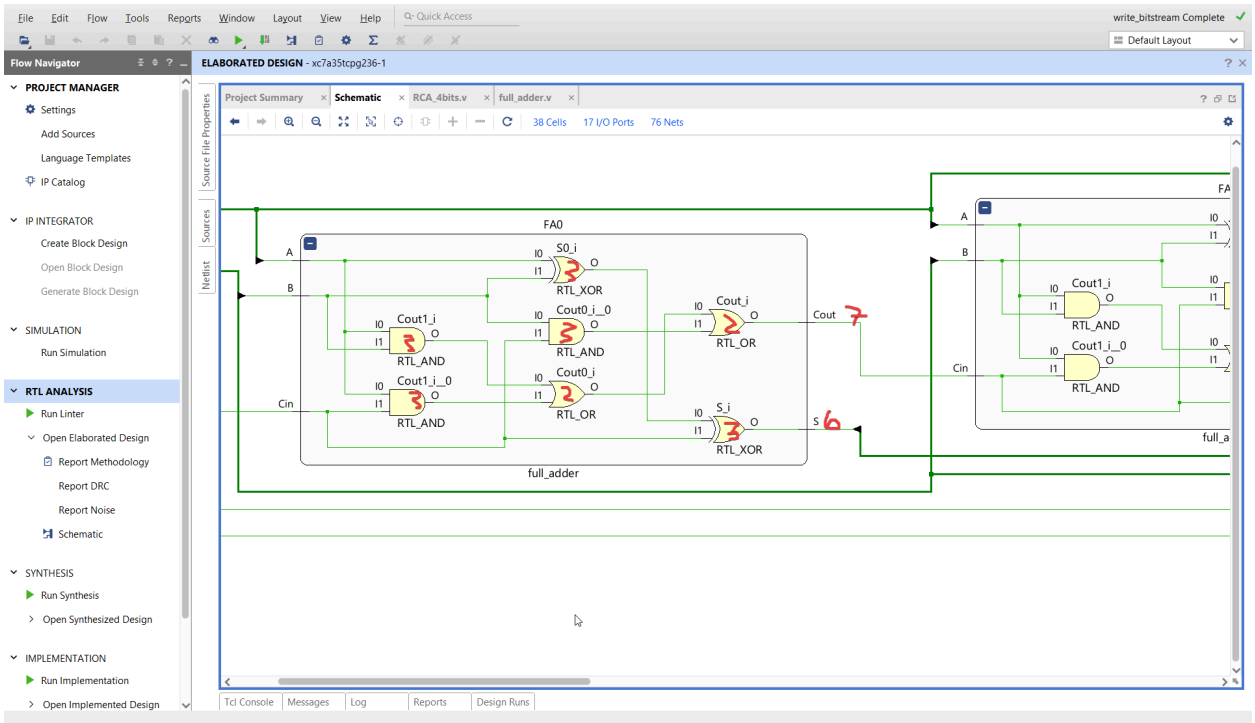
CLA:



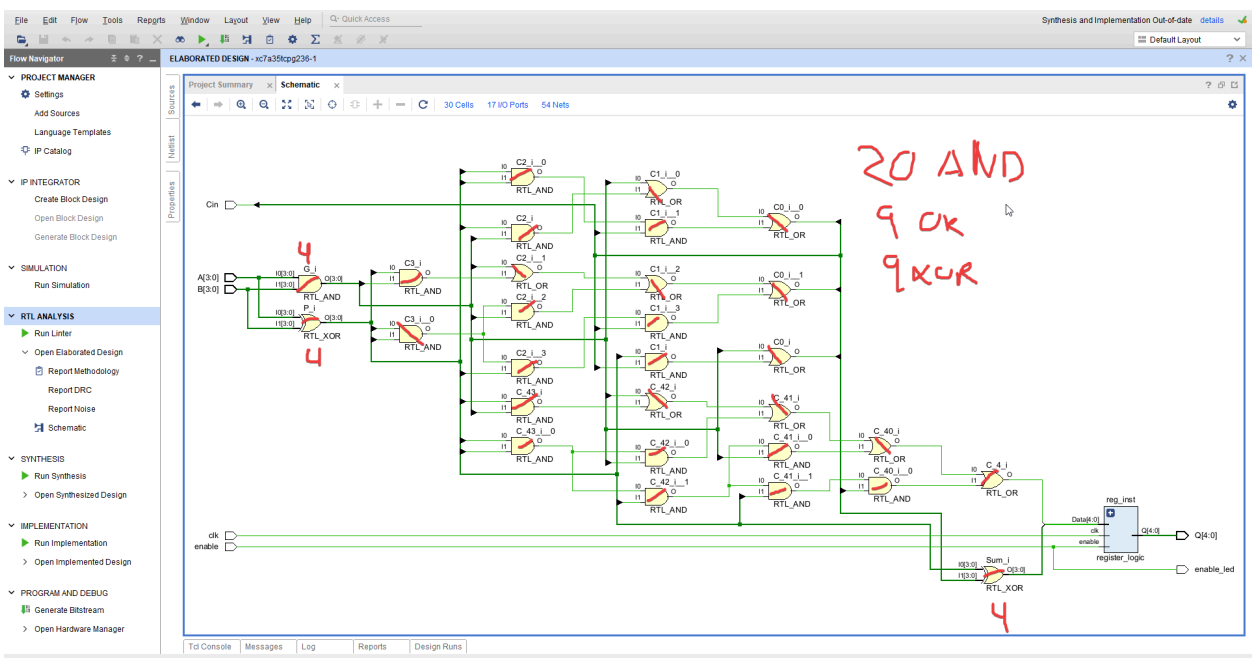
xiii. Delay and area calculations for both adder techniques showing all work

RCA: 4 full adders, each full adder uses 3 ANDS, 2 ORS, 2 XORS, so total of 12 ANDS, 8 ORS, 8 XORS. This comes to an area of 12*4+8*4+8*6=128 transistors. For one full

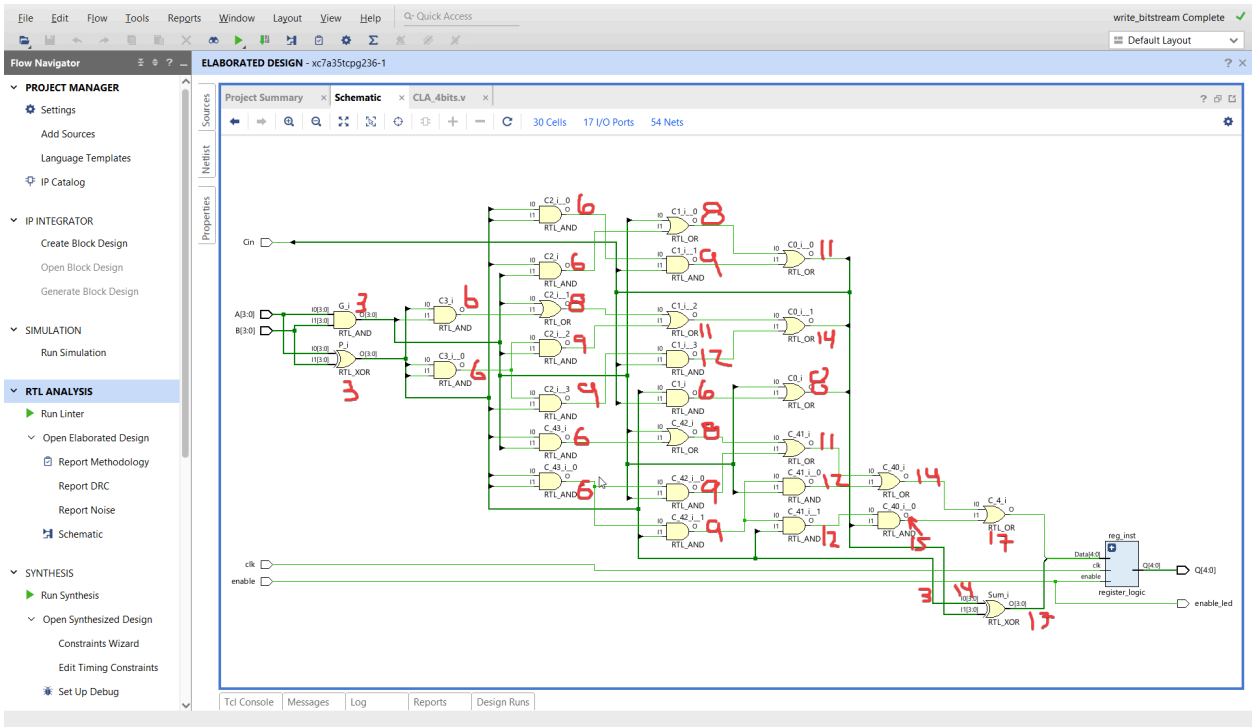
adder the sum will take 6ns and the cout will take 7ns. Since the next full adder depends on the last we can say the sum will take 24ns and the cout will take 28ns for all four. So the critical path is for the couts with 28ns for the full answer.



CLA: 20 ANDS, 9 ORS, 9 XORS -> 20*4+9*4+9*6=170 transistors



It appears that the critical path is 17ns as seen in the image below



xiv. Brief conclusion regarding the pros and cons of each of the techniques

The CLA is more time efficient, at 11ns faster for 4 bits with the current implementation, but takes many more transistors. The RCA is much smaller, and requires fewer gates/transistors but takes longer as it is cascading, not parallel in execution.